Лабораторийн ажил №2

Сэдэв: Линуксийн цөмийн модуль (Linux Kernel Modules)

Хичээл: Үйлдлийн систем

Гүйцэтгэсэн: Н.Мөнхпүрэв, 23B1NUM0114

Огноо: 2026-10-02

1. ОРШИЛ

Энэ лабораторийн ажилд Линуксийн цөмд ачаалагддаг энгийн модуль бичиж, `insmod`, `rmmod`, `lsmod`, `dmesg` командуудаар ачаалж, устгаж, шалгав. Ажлыг гурван хэсэгт хийсэн: (1) `simple.c` модуль, (2) цөмийн холбоост жагсаалт ашигласан `birthday_list.c` модуль, (3) жагсаалтыг `/proc/birthdays` файлаар хэрэглэгчийн түвшинд харуулдаг `birthday_proc.c` модуль.

Цөмийн код нь хэрэглэгчийн программаас ялгаатай нь алдаа гарвал бүх системд нөлөөлдөг тул санах ой авах, чөлөөлөх, модулийг зөв устгахад онцгой анхаарсан. Тайланд онолын товч тайлбар, кодын тайлбар, ажиллуулах дараалал болон гарах үр дүнг оруулсан бөгөөд бүрэн эх кодыг хавсралтад өгөв.

2. ЗОРИЛГО

Цөмийн модуль хэрхэн бичигдэж, компайл хийгдэж, цөмд ачаалагдан устдагийг энгийн жишээгээр харуулах, мөн цөмийн `list_head` холбоост жагсаалт болон `procfs`-ийн хэрэглээг тайлбарлах.

Зорилтууд:

1) `simple.c` модулийг компайл хийж, ачаалж, устгаад `dmesg`-ээр шалгах.

2) 5 элементтэй `struct birthday` холбоост жагсаалтыг `init` функцэд үүсгэж, `exit` функцэд бүгдийг нь `kfree()`-ээр чөлөөлөх.

3) `birthday_proc.c` модулиар 2000 оноос өмнөх огноог тэмдэглэж, `/proc/birthdays` файлаар унших.

4) `list_add()` ба `list_add_tail()`-ийн ялгааг гаралтаар харуулж, хязгаарлалтыг дүгнэх.

3. ОНОЛЫН СУДАЛГАА

3.1 Онол

Цөмийн модуль (kernel module) нь системийг дахин ачаалахгүйгээр ажиллаж байгаа цөмд нэмж, хасаж болдог код юм [1]. Модуль нь цөмийн функцийг шууд дуудаж чаддаг боловч цөмийн эрхээр ажилладаг тул алдаа нь системийг гацаах эрсдэлтэй [2].

Эхлэх ба гарах функц. Модуль ачаалагдахад `module_init()`-ээр бүртгэсэн функц, устахад `module_exit()`-ээр бүртгэсэн функц дуудагдана. Эхлэх функц `int` буцааж, 0 бол амжилттай, сөрөг утга бол алдаа гэж үзэн модулийг ачаалахгүй. Гарах функц `void` бөгөөд параметргүй [2]. Энэ ажилд эдгээр нь `simple_init()`, `simple_exit()` зэрэг функцууд.

`printk()` ба `dmesg`. `printk()` нь цөмийн `printf()` юм. Гаралт нь дэлгэц рүү биш цөмийн лог буферт орох тул `dmesg` командаар уншина. `KERN_INFO` нь мэдээллийн түвшин, `KERN_ERR` нь алдааны түвшин заана [2].

Компайл хийх. Модулийг `obj-m += simple.o` гэсэн мөртэй `Makefile`-аар цөмийн build системээр (kbuild) компайл хийнэ. Үр дүнд нь `simple.ko` файл үүснэ [3]. Ачаалахдаа `sudo insmod simple.ko`, жагсаалтыг `lsmod`, устгахдаа `sudo rmmod simple` ашиглана.

Цөмийн холбоост жагсаалт. Линуксийн цөм `<linux/list.h>` дотор дугуй, хоёр талт холбоост жагсаалт (circular doubly linked list) өгдөг. Өгөгдлийн бүтэц дотроо `struct list_head list;` гишүүн нэмснээр тухайн бүтэц жагсаалтын элемент болно. `list_head` нь `next`, `prev` гэсэн хоёр заагчтай [1]. Ашигласан үндсэн макронууд:

```
LIST_HEAD(name);                  /* хоосон жагсаалтын толгой */
list_add(&p->list, &name);        /* толгой дээр нэмэх */
list_add_tail(&p->list, &name);   /* сүүлд нэмэх */
list_for_each_entry(ptr, &name, list)         /* гүйх */
list_for_each_entry_safe(ptr, next, &name, list) /* устгахад гүйх */
list_del(&p->list);               /* жагсаалтаас салгах */
```

Цөмийн санах ой. `kmalloc(size, GFP_KERNEL)` нь хэрэглэгчийн `malloc()`-тай төстэй цөмийн санах ой авна, `kfree()` нь чөлөөлнө. Чөлөөлөхгүй орхивол модуль устсан ч санах ой цөмд үлдэж memory leak болно.

`procfs`. `/proc` нь цөмийн доторх мэдээллийг файл мэт уншдаг виртуал файлын систем юм. `proc_create()`-ээр файл үүсгэж, `seq_file` (`single_open`, `seq_printf`)-ээр агуулгыг гаргана. Модулиа устгахдаа `proc_remove()` хийнэ. Шинэ цөмд (5.6-аас хойш) файлын үйлдлийг `struct proc_ops`-оор тодорхойлдог.

3.2 Судалгаа

Асуулт: жагсаалтад элемент нэмэх `list_add()` ба `list_add_tail()`-ийн ялгаа юу вэ, мөн устгахдаа яагаад энгийн `list_for_each_entry()` биш `list_for_each_entry_safe()` хэрэглэх вэ?

Хүснэгт 1. list_add ба list_add_tail-ийн харьцуулалт

Шалгуур	**list_add()**	**list_add_tail()**
Байрлал	Толгойн дараа (эхэнд)	Толгойн өмнө (төгсгөлд)
Дараалал	Сүүлд нэмсэн нь эхэнд гарна (LIFO, stack)	Нэмсэн дарааллаараа (FIFO, queue)
Энэ ажилд	birthday_proc.c-ийн эхний 2 огноо	birthday_list.c-ийн 5 огноо, birthday_proc.c-ийн сүүлийн 2 огноо

Хүснэгт 1-ээс харахад дарааллыг хадгалах хэрэгтэй үед `list_add_tail()`, хамгийн сүүлд орсныг түрүүлж авах хэрэгтэй үед `list_add()` тохирно. Энэ ажлын 2-р хэсэгт огноог оруулсан дарааллаар нь харуулах учир `list_add_tail()` сонгосон. 3-р хэсэгт хоёр аргыг нэг жагсаалтад хэрэглэж ялгааг нь харуулсан.

Устгах үед `kfree(ptr)` хийсний дараа `ptr->list.next`-ийг унших нь чөлөөлсөн санах ойд хандах болно. `list_for_each_entry_safe()` нь дараагийн элементийг `next` хувьсагчид өмнө нь хадгалдаг тул элементийг устгасан ч гүйлт зөв үргэлжилнэ [3]. Тиймээс устгах давталтад `_safe` хувилбарыг сонгосон.

4. ХЭРЭГЖҮҮЛЭЛТ

Гурван модулийг тус тусад нь бичиж, нэг `Makefile`-аар компайл хийв. Модуль бүр ачаалагдах, устахад `printk()`-ээр лог бичдэг тул үр дүнг `dmesg`-ээр шалгана. Бүрэн эх кодыг Хавсралт А-д үзүүлэв.

4.1 Шийдэл ба кодын тайлбар

Ролт: модулиудад хэрэглэгчийн ролт байхгүй, өгөгдлийг кодод тогтмол оруулсан. Гаралт: цөмийн лог (`dmesg`) болон `/proc/birthdays` файл.

I хэсэг. simple.c

`simple_init()` нь ачаалахад "Loading Module", `simple_exit()` нь устгахад "Removing Module" гэж лог бичнэ. Зааварт өгсөн кодоос зөвхөн функцүүдийг `static`, `__init`, `__exit` гэж тэмдэглэсэн; ажиллах зарчим нь адил.

Код 1. simple.c-ийн үндсэн хэсэг

```
static int __init simple_init(void)
{
     printk(KERN_INFO "Loading Module\n");
     return 0;
}

static void __exit simple_exit(void)
{
     printk(KERN_INFO "Removing Module\n");
}

module_init(simple_init);
```

```
module_exit(simple_exit);
```

II хэсэг. birthday_list.c

`struct birthday` нь өдөр, сар, жил болон `list_head` агуулна. `birthday_init()` дотор давталтаар 5 удаа `kmalloc()` хийж, огноог `(i+1)*2` өдөр, `8+i` сар, `1995+i` жил гэж онооно (2/8/1995 ... 10/12/1999). Элемент бүрийг `list_add_tail()`-аар нэмж, дараа нь `list_for_each_entry()`-ээр бүгдийг лог руу хэвлэнэ.

Код 2. birthday_init дахь элемент үүсгэх давталт

```
for (i = 0; i < 5; i++) {
     person = kmalloc(sizeof(*person), GFP_KERNEL);
     if (!person) {
           printk(KERN_ERR "Birthday Module: kmalloc failed!\n");
           return -ENOMEM;
     }
     person->day = (i + 1) * 2;
     person->month = 8 + i;
     person->year = 1995 + i;
     INIT_LIST_HEAD(&person->list);
     list_add_tail(&person->list, &birthday_list);
}
```

`birthday_exit()` нь `list_for_each_entry_safe()`-ээр гүйж, элемент бүрийг лог руу хэвлээд `list_del()`-ээр салгаж, `kfree()`-ээр чөлөөлнө.

Код 3. birthday_exit дахь устгах давталт

```
list_for_each_entry_safe(ptr, next, &birthday_list, list) {
     printk(KERN_INFO "  deleting: %02d/%02d/%04d\n",
           ptr->day, ptr->month, ptr->year);
     list_del(&ptr->list);
     kfree(ptr);
}
```

III хэсэг. birthday_proc.c

Энэ модуль 4 огноотой. Эхний хоёрыг (1/1/1995, 12/5/1998) `list_add()`-аар, сүүлийн хоёрыг (3/7/2001, 20/9/2005) `list_add_tail()`-аар нэмсэн. `list_add()` толгой дээр нэмдэг тул эхний хоёр нь урвуу дарааллаар орно: жагсаалт 12/5/1998 → 1/1/1995 → 3/7/2001 → 20/9/2005 болно. `proc_create()` нь `/proc/birthdays` файл үүсгэнэ. Уншихад `birthdays_show()` ажиллаж, 2000 оноос өмнөх огноог `*` ба `(pre-2000)`, бусдыг `-` тэмдгээр хэвлэнэ.

Код 4. /proc/birthdays-ийн гаралт гаргах функц

```
static int birthdays_show(struct seq_file *file, void *v)
{
     struct birthday *ptr;

     seq_puts(file, "== Birthday List ==\n");
     list_for_each_entry(ptr, &birthday_list, list) {
           seq_printf(file, "%c %02d/%02d/%04d%s\n",
                 ptr->year < 2000 ? '*' : '-',
```

```
                ptr->day, ptr->month, ptr->year,
                ptr->year < 2000 ? " (pre-2000)" : "");
    }
    return 0;
}
```

`rmmod` хийхэд эхлээд `proc_remove()`-ээр файлыг устгаж, дараа нь жагсаалтыг `_safe` давталтаар цэвэрлэнэ. Ингэснээр модуль устсаны дараа файл уншигдах боломжгүй болно.

Makefile-д гурван модулийг `obj-m += ...` гэж бүгдийг нь жагсаасан тул нэг `make` команд гурвыг компайл хийнэ. `make run` нь модуль бүрийг ачаалж, шалгаад устгадаг.

4.2 Туршилт ба үр дүн

Ажиллуулах орчин: Линукс, цөмийн толгой файл (`linux-headers`) суусан байх ёстой; хувилбарыг `uname -r` командаар шалгана. Компайл хийхэд `make`, ачаалахад `sudo insmod`, шалгахад `lsmod` ба `dmesg` ашиглана.

Код 5. Туршилтын командууд

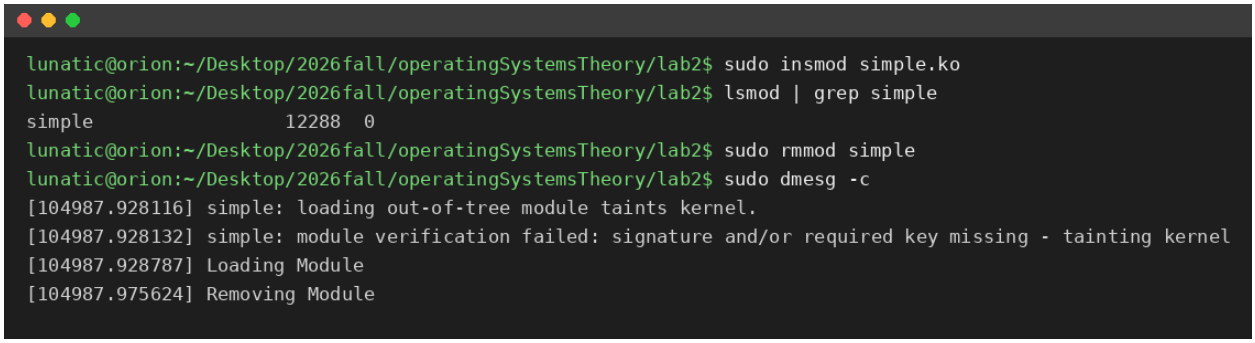
```
make
sudo dmesg -c

sudo insmod simple.ko
lsmod | grep simple
dmesg | tail
sudo rmmod simple
dmesg | tail

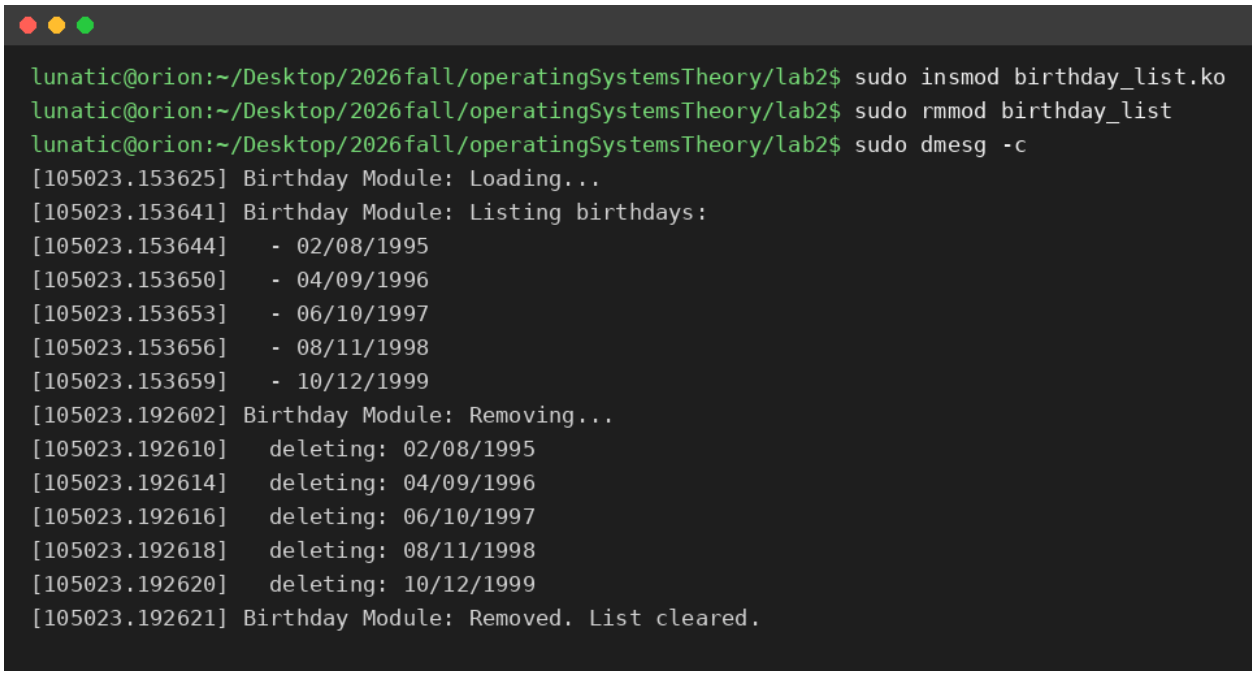
sudo insmod birthday_list.ko
lsmod | grep birthday_list
dmesg | tail -n 10
sudo rmmod birthday_list
dmesg | tail -n 10

sudo insmod birthday_proc.ko
cat /proc/birthdays
sudo rmmod birthday_proc
dmesg | tail -n 10
```

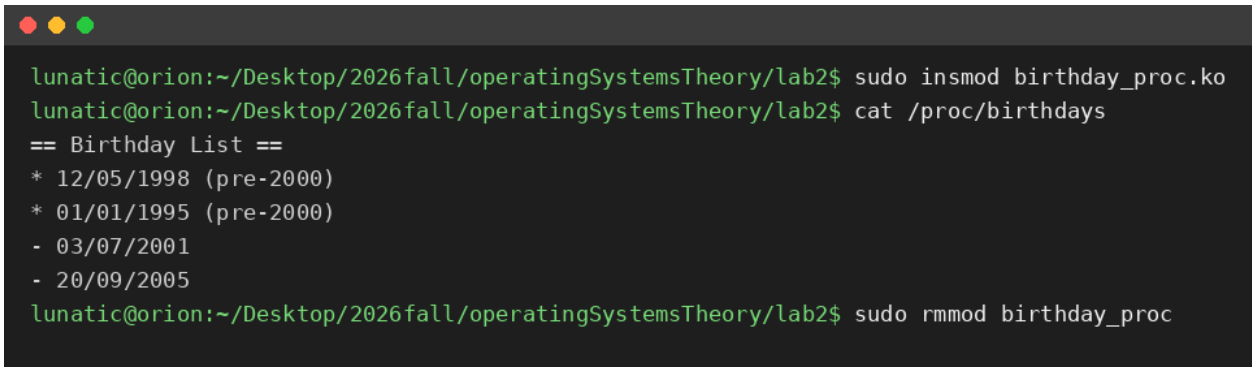
Модуль бүрийг VM дээр ажиллуулж, терминалын гаралтыг Зураг 1–3-т үзүүлэв. Хүснэгт 2-т гаралтыг хураангуйлав. `dmesg` нь мөр бүрийн өмнө цагийн тэмдэг (`[104987.928787]`) нэмж харуулдаг.



Зураг 1. simple модулийг ачаалж, устгасан үеийн терминалын гаралт



Зураг 2. birthday_list модулийг ачаалж, устгасан үеийн dmesg-ийн гаралт



Зураг 3. birthday_proc модулийг ачаалж, /proc/birthdays уншсан гаралт

Хүснэгт 2. Модуль бүрийн гаралт

№	Үйлдэл	Гарсан гаралт
1	insmod simple.ko lsmod | grep simple	`simple  12288  0` `Loading Module`
2	rmmod simple	`Removing Module`

№	Үйлдэл	Гарсан гаралт

3	insmod birthday_list.ko	`Birthday Module: Loading...`
		`Birthday Module: Listing birthdays:`
		`  - 02/08/1995`
		`  - 04/09/1996`
		`  - 06/10/1997`
		`  - 08/11/1998`
		`  - 10/12/1999`

4	rmmod birthday_list	`Birthday Module: Removing...`
		`  deleting: 02/08/1995`
		`  deleting: 04/09/1996`
		`  deleting: 06/10/1997`
		`  deleting: 08/11/1998`
		`  deleting: 10/12/1999`
		`Birthday Module: Removed. List cleared.`

5	insmod birthday_proc.ko
	cat /proc/birthdays
		`== Birthday List ==`
		`* 12/05/1998 (pre-2000)`
		`* 01/01/1995 (pre-2000)`
		`- 03/07/2001`
		`- 20/09/2005`

6	rmmod birthday_proc	`Алдаа гараагүй`

Үр дүнгийн тайлбар: Бүх гаралт кодоос хүлээсэн үр дүнтэй таарсан. 3-р туршилтад огноонууд `list_add_tail()`-ээр нэмсэн дарааллаараа гарсан. 5-р туршилтад 12/05/1998 нь 01/01/1995-аас өмнө гарсан нь `list_add()` толгой дээр нэмдгийг харуулна, харин 2001, 2005 оны огноо `list_add_tail()`-ийн дагуу сүүлд, дарааллаараа байна. 4-р туршилтад устгах дараалал нь жагсаалтын дарааллыг дагасан. Эдгээр нь сонгосон тохиолдлуудын үр дүн бөгөөд бүх нөхцөлд зөв ажиллана гэдгийг батлахгүй.

Ажиглалт: `simple` модулийг ачаалахад "module verification failed: signature and/or required key missing - tainting kernel" гэсэн мэдээлэл гарсан. Энэ нь модуль гарын үсэггүй, цөмийн гаднах (out-of-tree) модуль тул цөмийг "tainted" гэж тэмдэглэсэн хэвийн анхааруулга бөгөөд модуль зөв ажилласан. Мөн `dmesg`-ийг `sudo`-гүй ажиллуулахад "read kernel buffer failed: Operation not permitted" гарсан тул лог унших үед `sudo dmesg` хэрэглэсэн. birthday_proc модулийг устгах үеийн логийг (Removing, deleting мөрүүд) хадгалж аваагүй тул Хүснэгт 2-т оруулаагүй.

Түгээмэл алдаа: `Operation not permitted` гарвал `sudo`-гүй ажиллуулсан; `Invalid module format` гарвал `uname -r` ба суусан толгой файлын хувилбар таарахгүй байна [2].

5. ДҮГНЭЛТ

Зорилгын дагуу цөмийн модулийг бичиж, компайл хийж, `insmod`/`rmmod`-оор ачаалж устгах дарааллыг гурван модулиар харуулав. `birthday_list` модуль 5 элементтэй жагсаалтыг үүсгэж, устгахдаа `list_for_each_entry_safe()`, `list_del()`, `kfree()`-ээр бүгдийг чөлөөлдөг. `birthday_proc` модуль нь `list_add()` ба `list_add_tail()`-ийн дарааллын ялгааг `/proc/birthdays` файлаар хэрэглэгчийн түвшинд харуулдаг.

Давуу тал нь цөмийн бэлэн `list_head` бүтцийг ашигласнаар жагсаалтыг өөрөө бичих шаардлагагүй болсон. Хязгаарлалт: `init` функцэд `kmalloc()` эсвэл `proc_create()` амжилтгүй болбол өмнө нь авсан элементүүд чөлөөлөгдөхгүй (модуль ачаалагдаагүй тул `exit` дуудагдахгүй) тул memory leak гарна. Мөн жагсаалтыг зэрэгцээ хандалтаас хамгаалах түгжээ хийгээгүй. Цаашид алдааны үед цэвэрлэх хэсэг (cleanup) болон түгжээ нэмж сайжруулж болно.

6. АШИГЛАСАН МАТЕРИАЛ

[1] Silberschatz A., Galvin P. B., Gagne G. Operating System Concepts, 10th ed. Wiley, 2018. 1-р бүлэг (цөмийн өгөгдлийн бүтэц), 2-р бүлэг (ачаалагдах цөмийн модуль).

[2] Лаб №2 - Линуксийн цөмийн модуль (Linux Kernel Modules). Үйлдлийн систем хичээлийн лабораторийн заавар (PDF).

[3] The Linux Kernel documentation. Building External Modules. https://docs.kernel.org/kbuild/modules.html . Хандсан: 2026-10-02.

7. ХАВСРАЛТ

Бүрэн эх кодыг доор файл бүрээр нь үзүүлэв.

Хавсралт А. simple.c

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>

static int __init simple_init(void)
{
     printk(KERN_INFO "Loading Module\n");
     return 0;
}

static void __exit simple_exit(void)
{
     printk(KERN_INFO "Removing Module\n");
}

module_init(simple_init);
module_exit(simple_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Simple Module");
MODULE_AUTHOR("SGG");
```

Хавсралт Б. birthday_list.c

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/list.h>
#include <linux/module.h>
#include <linux/slab.h>
```

```
struct birthday {
     int day;
     int month;
     int year;
     struct list_head list;
};

static LIST_HEAD(birthday_list);

static int __init birthday_init(void)
{
     struct birthday *person;
     int i;

     printk(KERN_INFO "Birthday Module: Loading...\n");

     for (i = 0; i < 5; i++) {
           person = kmalloc(sizeof(*person), GFP_KERNEL);
           if (!person) {
                 printk(KERN_ERR "Birthday Module: kmalloc failed!\n");
                 return -ENOMEM;
           }

           person->day = (i + 1) * 2;
           person->month = 8 + i;
           person->year = 1995 + i;
           INIT_LIST_HEAD(&person->list);
           list_add_tail(&person->list, &birthday_list);
     }

     {
           struct birthday *ptr;

           printk(KERN_INFO "Birthday Module: Listing birthdays:\n");
           list_for_each_entry(ptr, &birthday_list, list) {
                 printk(KERN_INFO "  - %02d/%02d/%04d\n",
                       ptr->day, ptr->month, ptr->year);
           }
     }

     return 0;
}

static void __exit birthday_exit(void)
{
     struct birthday *ptr;
     struct birthday *next;

     printk(KERN_INFO "Birthday Module: Removing...\n");

     list_for_each_entry_safe(ptr, next, &birthday_list, list) {
           printk(KERN_INFO "  deleting: %02d/%02d/%04d\n",
                 ptr->day, ptr->month, ptr->year);
           list_del(&ptr->list);
           kfree(ptr);
     }

     printk(KERN_INFO "Birthday Module: Removed. List cleared.\n");
```

```
}

module_init(birthday_init);
module_exit(birthday_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Kernel Linked List Example (Birthdays)");
MODULE_AUTHOR("Batbayar");
```

Хавсралт B. birthday_proc.c

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/list.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/slab.h>

#define PROC_NAME "birthdays"

struct birthday {
     int day;
     int month;
     int year;
     struct list_head list;
};

static LIST_HEAD(birthday_list);
static struct proc_dir_entry *proc_entry;

static int birthdays_show(struct seq_file *file, void *v)
{
     struct birthday *ptr;

     seq_puts(file, "== Birthday List ==\n");
     list_for_each_entry(ptr, &birthday_list, list) {
           seq_printf(file, "%c %02d/%02d/%04d%s\n",
                 ptr->year < 2000 ? '*' : '-',
                 ptr->day, ptr->month, ptr->year,
                 ptr->year < 2000 ? " (pre-2000)" : "");
     }

     return 0;
}

static int birthdays_open(struct inode *inode, struct file *file)
{
     return single_open(file, birthdays_show, NULL);
}

static const struct proc_ops birthdays_fops = {
     .proc_open = birthdays_open,
     .proc_read = seq_read,
     .proc_lseek = seq_lseek,
     .proc_release = single_release,
};

static int __init birthday_init(void)
```

```
{
     struct birthday *p;

     printk(KERN_INFO "Birthday Proc Module: Loading...\n");

     /* Толгой дээр нэмэх (list_add) */
     p = kmalloc(sizeof(*p), GFP_KERNEL);
     if (!p)
           return -ENOMEM;
     p->day = 1;
     p->month = 1;
     p->year = 1995;
     INIT_LIST_HEAD(&p->list);
     list_add(&p->list, &birthday_list);

     p = kmalloc(sizeof(*p), GFP_KERNEL);
     if (!p)
           return -ENOMEM;
     p->day = 12;
     p->month = 5;
     p->year = 1998;
     INIT_LIST_HEAD(&p->list);
     list_add(&p->list, &birthday_list);

     /* Сүүлд нэмэх (list_add_tail) */
     p = kmalloc(sizeof(*p), GFP_KERNEL);
     if (!p)
           return -ENOMEM;
     p->day = 3;
     p->month = 7;
     p->year = 2001;
     INIT_LIST_HEAD(&p->list);
     list_add_tail(&p->list, &birthday_list);

     p = kmalloc(sizeof(*p), GFP_KERNEL);
     if (!p)
           return -ENOMEM;
     p->day = 20;
     p->month = 9;
     p->year = 2005;
     INIT_LIST_HEAD(&p->list);
     list_add_tail(&p->list, &birthday_list);

     /* procfs үүсгэх: /proc/birthdays */
     proc_entry = proc_create(PROC_NAME, 0, NULL, &birthdays_fops);
     if (!proc_entry) {
           printk(KERN_ERR "Failed to create /proc/%s\n", PROC_NAME);
           return -ENOMEM;
     }

     printk(KERN_INFO "Birthday Proc Module: /proc/%s created\n",
           PROC_NAME);
     return 0;
}

static void __exit birthday_exit(void)
{
     struct birthday *ptr;
     struct birthday *next;
```

```
    printk(KERN_INFO "Birthday Proc Module: Removing...\n");

    /* procfs устгах */
    if (proc_entry)
        proc_remove(proc_entry);

    /* Жагсаалтыг цэвэрлэх */
    list_for_each_entry_safe(ptr, next, &birthday_list, list) {
        printk(KERN_INFO "  deleting: %02d/%02d/%04d\n",
               ptr->day, ptr->month, ptr->year);
        list_del(&ptr->list);
        kfree(ptr);
    }

    printk(KERN_INFO "Birthday Proc Module: Removed.\n");
}

module_init(birthday_init);
module_exit(birthday_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Batbayar");
MODULE_DESCRIPTION("Birthday list with procfs + list_add vs list_add_tail
demo");
```

Хавсралт Г. Makefile

```
obj-m += simple.o
obj-m += birthday_list.o
obj-m += birthday_proc.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

.PHONY: all run clean

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

run: all
    @echo "=== removing old lab modules if loaded ==="
    @sudo rmmod birthday_proc 2>/dev/null || true
    @sudo rmmod birthday_list 2>/dev/null || true
    @sudo rmmod simple 2>/dev/null || true
    @echo "=== simple ==="
    sudo insmod simple.ko
    lsmod | grep simple
    sudo rmmod simple
    @echo "=== birthday_list ==="
    sudo insmod birthday_list.ko
    lsmod | grep birthday_list
    sudo rmmod birthday_list
    @echo "=== birthday_proc ==="
    sudo insmod birthday_proc.ko
    lsmod | grep birthday_proc
    cat /proc/birthdays
    sudo rmmod birthday_proc
    @echo "=== dmesg ==="
```

```
	sudo dmesg | tail -n 40

clean:
	$(MAKE) -C $(KDIR) M=$(PWD) clean
```